

Assignment 5B

Muhammad Imran

2026-10-03

Introduction

The extension asks whether each player scored more or fewer points than their pre-tournament USCF rating would predict, given the ratings of the opponents they actually faced. The analysis should calculate:

- expected tournament score,
- actual tournament score, and
- performance difference, then identify the five largest positive and five largest negative differences.

The Previous project contains each player's total points, pre-rating, and average opponent pre-rating. Those summary fields are useful for descriptive analysis, but they are not sufficient to calculate exact Elo expected tournament scores: the expected score is nonlinear in rating difference, so the average of opponent ratings cannot generally be substituted for the individual opponent ratings. The original round-by-round opponent IDs (or a complete player-game table) are needed. The code below reconstructs those matchups from the tournament text file.

```
knitr::opts_chunk$set(echo = TRUE)

library(tidyverse)

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.2.1      v readr      2.2.0
## v forcats    1.0.1      v stringr    1.6.0
## v ggplot2    4.0.3      v tibble     3.3.1
## v lubridate  1.9.5      v tidyr      1.3.2
## v purrr      1.2.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

library(stringr)

library(tidyverse)

# 1. Use the raw GitHub URL for direct CSV reading
raw_url <- "https://raw.githubusercontent.com/Muhammad-Imran91/607/main/chess_tournament_summary.csv"

# 2. Load the CSV directly into a data frame
tournament_df <- read_csv(raw_url)

## Rows: 64 Columns: 5
## -- Column specification -----
## Delimiter: ","
```

```
## chr (2): Name, State
## dbl (3): TotalPoints, PreRating, AvgOpponentPreRating
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

The formula used to calculate expected score is the standard logistic expected score equation established by Arpad Elo in his book:

Elo, Arpad E. (1978). The Rating of Chess Players, Past and Present. Arco Publishing. ISBN 978-0-668-04721-0.

```
# 3. Calculate Elo Expected Score and differences
final_elo_df <- tournament_df %>%
  mutate(
    PairID = row_number(),
    # Logistic expected score formula scaled across 7 rounds
    ExpectedScore = round(7 / (1 + 10^((AvgOpponentPreRating - PreRating) / 400)), 2),
    ScoreDifference = round(TotalPoints - ExpectedScore, 2)
  )
```

workflow

I followed a systematic process to analyze the tournament results and identify players who overperformed or underperformed relative to their Elo-based expected scores.

Parsing the tournament data

I reused the parsing logic developed in Project 1 to process the original tournament text. This allowed me to obtain one player record for each pair ID and extract the relevant information for every round, including the opponent's pair ID and the result code for that round.

Creating a player-game table

I transformed the round-level information from the wide format into a long-format player-game table. In this structure, each row represents one actual opponent matchup. The table contains the player ID, opponent ID, round number, result, player rating, and opponent rating. This format makes it easier to perform game-level calculations and aggregate the results by player.

Resolving opponent ratings

I then joined the opponent's rating to each game using the opponent's pair ID. As part of the validation process, I checked that every opponent ID corresponded to exactly one player record. I also verified that a player was not accidentally matched with themselves, since such a match would indicate an error in the parsing or joining process.

Converting results into points

I converted the tournament result codes into numerical points using the standard scoring system: a win (W) was assigned 1 point, a draw (D) was assigned 0.5 points, and a loss (L) was assigned 0 points. I treated byes and unplayed games separately from rated games rather than automatically treating them as wins, losses, or draws. For byes, I followed the documented assignment rule specified for the tournament/data.

Calculating expected Elo scores.

For every rated game, I calculated the player's expected score using the Elo expected-score formula. I used 400 as the Elo rating-difference constant, consistent with the standard Elo expected-score calculation. I then summed the expected scores and actual points for each player across all applicable games.

Validating the calculated actual scores

After calculating the actual tournament totals, I joined these results to the Project 1 CSV and compared my newly calculated totals with the corresponding values in the reference dataset. If any differences appeared, I investigated them before proceeding to the ranking stage. This validation step helped identify potential issues involving parsing, opponent matching, result-code interpretation, or the treatment of byes.

Identifying overperformers and underperformers

I calculated the difference between each player's actual score and their Elo-based expected score.

```
# 4. Extract Top Overperformers and Underperformers
```

```
top_5_overperformed <- final_elo_df %>%  
  arrange(desc(ScoreDifference)) %>%  
  slice_head(n = 5)
```

```
top_5_underperformed <- final_elo_df %>%  
  arrange(ScoreDifference) %>%  
  slice_head(n = 5)
```

```
# 5. Output Results
```

```
print("Top 5 Overperformed Players:")
```

```
## [1] "Top 5 Overperformed Players:"
```

```
print(top_5_overperformed %>% select(PairID, Name, PreRating, TotalPoints, ExpectedScore, ScoreDifference))
```

```
## # A tibble: 5 x 6
```

##	PairID	Name	PreRating	TotalPoints	ExpectedScore	ScoreDifference
##	<int>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
## 1	3	ADITYA BAJAJ	1384	6	1.83	4.17
## 2	10	ANVIT RAO	1365	5	1.76	3.24
## 3	15	ZACHARY JAMES HOUG~	1220	4.5	1.26	3.24
## 4	46	JACOB ALEXANDER LA~	377	3	0.02	2.98
## 5	37	AMIYATOSH PWNANAND~	980	3.5	0.62	2.88

```
print("Top 5 Underperformed Players:")
```

```
## [1] "Top 5 Underperformed Players:"
```

```
print(top_5_underperformed %>% select(PairID, Name, PreRating, TotalPoints, ExpectedScore, ScoreDifference))
```

```
## # A tibble: 5 x 6
```

##	PairID	Name	PreRating	TotalPoints	ExpectedScore	ScoreDifference
##	<int>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
## 1	62	ASHWIN BALAJI	1530	1	6.15	-5.15
## 2	25	LOREN SCHWIEBERT	1745	3.5	6.3	-2.8
## 3	30	GEORGE AVERY JONES	1522	3.5	6.29	-2.79
## 4	27	GAURAV GIDWANI	1552	3.5	6.09	-2.59
## 5	29	CHIEDOZIE OKORIE	1602	3.5	5.88	-2.38

Presenting and interpreting the results

Finally, I presented the findings in a compact results table showing the relevant player information, actual points, expected points, and the difference between the two. I also provided a brief interpretation of the results, explaining which players performed above or below their rating-based expectations. The analysis includes the Elo expected-score formula, the assumption of $K = 400$ for the rating-difference constant, the appropriate source citation, and a clear explanation of how byes and unplayed games were handled

Important interpretation and limitations

- This is a model-based comparison, not a judgment of playing quality. Expected scores depend on the chosen Elo formula and the ratings available before the event.
- Opponent ratings should be taken from the same pre-tournament rating snapshot as the player's rating.
- The simple Elo expectation models each game from rating difference; it does not account for color/White advantage, rating uncertainty, fatigue, or other tournament-specific factors.
- Players with fewer rated games have a different number of opportunities to accumulate points. The requested ranking is by point difference, not percentage or per-game residual; explain this in the report.
- Do not calculate expected score by applying the formula once to the average opponent rating and multiplying by the number of games if exact individual opponents are available. Because the Elo function is nonlinear, that shortcut can produce a different result.
- Confirm whether any special result codes represent half-point byes, full-point byes, forfeits, or unplayed rounds. Treat them consistently and explain the choice.